DNA read alignment and repeat detection

A DSA case study using suffix arrays, LCP arrays and the FM-index

DSA case study Python Four presenters

The problem and project scope

Many short patterns share one reference genome

Input: a DNA reference and short reads or query patterns.

Output: matching positions, strands, mismatch counts and repeats.

Repeated scans revisit the whole genome for every query.

Our prototype supports substitutions. It does not align insertions or deletions.

The five ADTs defined in the code

Abstract class	Concrete implementation	Purpose
SuffixArrayADT	SuffixArray	Ordered suffix positions
LCPArrayADT	LCPArray	Common-prefix lengths
BWTADT	BWT	Transform and inverse
FMIndexADT	FMIndex	Count and locate patterns
PatternMatcherADT	Four matcher classes	Shared search interface

Code organization and test data

Module	Responsibility
adt.py	Abstract contracts
data_structures.py	SA, LCP, BWT, FM-index and matchers
applications.py	Repeats, comparison and read alignment
genotype_driver.py	Seeded DNA, mutations, reads and FASTA
benchmark.py and tests/	Timing, invariants and reference checks

Naive search and KMP as baselines

Method	Approach	Worst-case time per query
Naive	Try each start position	O(nm)
KMP	Reuse the failure table	O(n + m)
Suffix array	Search an existing index	O(m log n + occ log occ)

All methods implement the same matcher interface. Index construction is a separate cost.

Suffix array representation

The list stores suffix starting positions in lexical order

Row	SA[row]	Suffix of banana$
0	6	$
1	5	a$
2	3	ana$
3	1	anana$
4	0	banana$
5	4	na$
6	2	nana$

Prefix doubling constructs the index

Initialize each suffix rank from its first character.

Sort suffix indices using a pair of ranks for two length-k blocks.

Assign new ranks to equal and unequal pairs.

Double k until every suffix has a distinct rank.

Implementation: _prefix_doubling() uses packed integer keys and Python comparison sorting.

Binary search finds a matching interval

Query	Matching SA rows	Text positions
ana	[2, 4)	[1, 3]
na	[5, 7)	[2, 4]
xyz	Empty interval	[]

bounds() finds the interval. count() measures it. search() sorts the text positions.

Why a suffix array fits this case study

Choice	Benefit	Trade-off
Suffix array	Reuses an index for many queries	Build cost before searching
Python list	Direct indexed access to SA and ranks	Python object memory overhead
Suffix tree alternative	Rich substring navigation	More complex node structure
KMP alternative	No genome index required	Rescans text per query

Suffix-array time and space

Operation	Bound	Reason
Construction	$O(n \log^2 n)$	O(log n) comparison-sort rounds
Count / interval	$O(m \log n)$	Two binary searches
Sorted positions	$O(m \log n + occ \log occ)$	Sort all reported positions
Construction space	O(n) words	Indices, ranks and keys

Query space with returned positions: $O(m + occ)$, including temporary string slices.

LCP array and Kasai’s algorithm

SA suffix	$	a$	ana$	anana$	banana$	na$	nana$
LCP	0	0	1	3	0	0	2

LCP[i] measures the shared prefix of suffixes at rows i−1 and i. Kasai builds it in O(n).

Repeat detection through the maximum LCP

banana contains ana at positions 1 and 3

Build the suffix array and LCP array.

Find the largest LCP value: 3.

Extract the matching prefix: ana.

Search that prefix to return every occurrence.

Overlapping occurrences count as repeats. Total worst-case time: $O(n \log^2 n)$.

Longest common substring of two genomes

One suffix array over A + # + B + $

Step	Reason
Join with a unique separator	Keep the sequence boundary explicit
Build SA and LCP	Order suffixes from both inputs
Check adjacent suffixes from different inputs	Find common prefixes across genomes
Keep the largest LCP	Return substring and positions in A and B

For input lengths a and b: $O((a + b) \log^2(a + b))$.

Approximate search with exact seeds

Pigeonhole principle for at most k substitutions

Pattern and budget	Pieces	Guarantee
ACGTAC, k = 1	ACG and TAC	At least one piece matches exactly
Candidate from a seed	hit − seed offset	Potential full-pattern start
Verification	Hamming distance ≤ k	Accept only valid candidates

A set removes duplicate candidates. If m ≤ k, the implementation scans all possible starts.

Read alignment on both DNA strands

Input read	Search sequence	Reported strand
Original sequence	Read itself	+
Opposite strand	Reverse complement	−

Each result stores (position, strand, mismatches). R reads require two approximate searches each.

Burrows–Wheeler Transform

BWT[row] = text[SA[row] − 1], with wraparound

Text	Suffix array	BWT
banana$	[6, 5, 3, 1, 0, 4, 2]	annb$aa

Construction takes O(n) given the suffix array. inverse() recovers banana$ using LF mapping.

FM-index backward search

Count occurrences without scanning the genome

Process ana right to left	Matching row interval
Initial interval	[0, 7)
Add a	[1, 4)
Add n	[5, 7)
Add a	[2, 4) = 2 occurrences

Update each bound with C[c] + Occ(c, bound).

FM-index storage and query trade-offs

Stored component	Purpose
BWT string	Backward-search text
C dictionary	Starting row of each character group
Occ checkpoints	Rank counts every B characters
Sampled SA dictionary	Recover positions with LF steps

Count: O(mB). Sorted locate: O(mB + occ·s·B + occ log occ).

Tests and recorded performance

Method	Recorded ms/query at n = 200,000
Naive	18.924
KMP	19.657
Suffix array	0.0128
FM-index	0.0291

Supplied benchmark: 100 queries of length 20. Timings depend on hardware and Python.

Live demonstration and project limits

```
python -m main.main demo
```

```
python -m main.main test
```

```
python -m main.main repeats --fasta data/sample_genome.fasta
```

Run from the project root. Explain build cost, overlapping matches and substitution-only alignment.